

CyberCase Intelligence Framework

Technical Documentation

Architecture · Frontend & Backend Stack · GraphRAG Pipeline · Database Schemas ·
Evaluation · Fine-Tuning · Follow-Up · Report Generation

Version 1.0 — June 12, 2026

Generated from source-code analysis of the repository

Table of Contents

1. Executive Overview.....	4
2. System Architecture.....	5
2.1 Service Topology.....	5
2.2 End-to-End Request Flow.....	5
2.3 Repository Layout.....	6
3. Technology Stack.....	7
3.1 Frontend.....	7
3.2 Backend Gateway.....	7
3.3 RAG / AI Stack.....	7
3.4 Models.....	8
4. Frontend (Next.js).....	9
4.1 Pages.....	9
4.2 API Client (src/lib/api.ts).....	9
4.3 Follow-Up Handling in the Chat UI.....	9
5. Backend API Gateway (FastAPI, port 8000).....	11
5.1 Application Setup (app/main.py).....	11
5.2 Configuration (app/config.py).....	11
5.3 Database Layer (app/database.py).....	11
5.4 REST Endpoints.....	11
5.5 Typhoon OCR Service (app/services/typhoon_ocr_reader.py).....	12
5.6 Design Rationale.....	12
6. RAG Service (FastAPI, port 8001).....	12
6.1 Startup: Shared Model Loading (app/main.py).....	12
6.2 Endpoints.....	12
7. GraphRAG Pipeline — Deep Dive.....	14
7.1 Typed Data Model (models.py).....	14
7.2 Ingestion Subsystem.....	14
7.3 Retrieval Subsystem.....	15
7.4 Context Builder (pipeline/context_builder.py).....	16
7.5 Cross-Lingual Layer (pipeline/cross_lingual.py).....	17
7.6 Query Router (pipeline/router.py).....	17
7.7 Linear Pipeline — GraphRAGChain (pipeline/chain.py).....	17
7.8 Agentic Pipeline — GraphRAGAgent (pipeline/agent_graph.py).....	18
8. Module: Context Evaluator — Self-Reflection (pipeline/evaluator.py).....	19
8.1 Verdicts and Result Shape.....	19
8.2 Judging Rules (system prompt).....	20
8.3 Slot Tracking (multi-turn interrogation).....	20
8.4 Fallback Strategies (retries exhausted).....	20
8.5 Safety Valves.....	20
9. Module: Follow-Up (interactive clarification).....	21
9.1 Full Sequence.....	21
9.2 QueryMerger (pipeline/query_merger.py).....	21
9.3 Properties Worth Noting.....	21
10. Module: Report Generation (pipeline/report_generator.py).....	22
10.1 CyberCaseReport Schema (7 sections).....	22
10.2 Schema-Enforced Generation.....	22
11. Database Schemas.....	23
11.1 PostgreSQL (application data).....	23
11.2 Neo4j (MITRE ATT&CK knowledge graph).....	23
11.3 Qdrant (vector store).....	24
12. Module: Evaluation (offline quality benchmarks).....	24

12.1 Retriever Benchmarks (retriever_metrics.py).....	24
12.2 Generation Benchmarks (generation_metrics.py).....	25
12.3 Ground Truth and Dataset Generation.....	25
12.4 Cross-Lingual Benchmark (crosslingual_benchmark.py).....	25
13. Module: Fine-Tune — MITRE ATT&CK Specialist (rag_service/finetune/).....	26
13.1 Workflow.....	26
13.2 Dataset Builder (data/build_dataset.py + templates.py).....	26
13.3 Training Configuration (ft_config.py, train_unsloth.py).....	26
13.4 A/B Comparison (compare/run_comparison.py).....	27
14. Deployment & Operations.....	28
14.1 Local Orchestration (docker-compose.yml).....	28
14.2 Secrets & Environment Variables.....	28
14.3 CI/CD (GitHub Actions → Docker Hub → Railway).....	28
14.4 Local Development Commands.....	28
15. Known Issues & Implementation Notes.....	30
16. Appendix: Key File Index.....	31

1. Executive Overview

CyberCase Intelligence Framework is a full-stack Retrieval-Augmented Generation (RAG) platform for analyzing cybersecurity incidents. It maps free-text incident descriptions (Thai or English) onto the **MITRE ATT&CK** knowledge base and produces plain-language explanations targeted at prosecutors and law-enforcement officers who have no cybersecurity background. The repository also bundles Thai legal source documents (Cybersecurity Act, PDPA, Computer Crime Act, Electronic Transactions Act) intended for the legal-analysis side of the platform.

The system is distinguished by four core capabilities:

- **GraphRAG hybrid retrieval** — every query is answered from two synchronized stores: a Qdrant vector database (semantic + lexical hybrid search over BGE-M3 embeddings) and a Neo4j knowledge graph (structured MITRE ATT&CK entities and relationships) whose subgraphs enrich the retrieved context.
- **Agentic self-reflection** — a LangGraph state machine evaluates whether retrieved context is sufficient; when it is not, the agent asks the user a targeted follow-up question, stores the answer as a structured incident fact, rewrites the query in MITRE-aligned vocabulary, and re-retrieves.
- **Cross-lingual operation** — Thai queries are translated to English for retrieval, but the original Thai query is retrieved in parallel (dual-query) so that one bad translation cannot sink retrieval. Answers are generated in English and translated back to Thai when needed.
- **Structured legal reporting** — a report generator produces a seven-section CyberCaseReport (case summary, indicators, MITRE mapping, justification, evidence to investigate, recommendations, system limitations) using schema-enforced LLM output.

This document describes the architecture, technology stack, database schemas, every major module (pipeline, evaluator, follow-up, report generation, evaluation, fine-tuning), and explains what the most important code blocks do.

2. System Architecture

2.1 Service Topology

The platform is split into three independently deployable services plus two cloud-hosted databases. Services communicate over HTTP; the backend acts as a thin gateway in front of the RAG service.

Service	Path	Port	Role
Frontend	frontend/	3000	Next.js single-page UI: landing page and chat interface with file upload.
Backend API	backend/	8000	FastAPI gateway. Owns PostgreSQL (users, health), Typhoon OCR for document uploads, CORS, and proxies all RAG calls to the RAG service via httpx.
RAG Service	rag_service/	8001	FastAPI service hosting the GraphRAG pipeline (LangGraph agent, LCEL chain, hybrid retriever, report generator). Loads BGE-M3 and the reranker once at startup.
PostgreSQL	Docker (local)	5433 → 5432	Relational store for application data (users). Async SQLAlchemy + asyncpg.
Neo4j	Neo4j Aura (cloud)	bolt/neo4j+s	Knowledge graph of MITRE ATT&CK entities and relationships.
Qdrant	Qdrant Cloud	https	Vector store with named dense + sparse vectors (hybrid search). Collections: mitre_entities, mitre_relationships.

Table 2-1: Services and data stores (ports from docker-compose.yml).

2.2 End-to-End Request Flow

```

Browser (Next.js, :3000)
  | POST /api/v1/rag/query      { query, use_agent }
  v
Backend Gateway (FastAPI, :8000)
  | httpx POST http://rag-service:8001/query (timeout 300 s)
  v
RAG Service (FastAPI, :8001)
  | GraphRAGAgent.query() -- LangGraph state machine
  |   route -> translate -> retrieve_multi -> evaluate
  |   sufficient -> reasoning -> (translate to Thai) -> END
  |   insufficient -> prepare_followup -> park session -> END
  |   broaden -> rewrite_query -> retrieve_multi (loop)
  v
Qdrant (hybrid vector search)      Neo4j (subgraph expansion)

```

Figure 2-1: Request path for an incident-analysis query.

When the agent needs clarification, the HTTP response carries `status="followup"`, a `followup_question` and a `session_id`. The frontend then sends the user's reply to `POST /api/v1/rag/resume` with the same session id, and the parked LangGraph state is resumed.

2.3 Repository Layout

Path	Contents
frontend/	Next.js 16 + React 19 + Tailwind CSS 4 app (landing + chat).
backend/	FastAPI gateway, SQLAlchemy models, Alembic scaffolding, Typhoon OCR service.
rag_service/app/	RAG FastAPI host (main.py) and the GraphRAG package.
rag_service/app/RAG/GraphRAG/	Pipeline, retrieval, ingestion, evaluation packages + config.py and models.py.
rag_service/finetune/	MITRE-specialist fine-tune module: dataset builder, QLoRA trainer, GGUF export, A/B comparison.
Mitre_ATT&CK Doc/	STIX 2.1 JSON bundles for enterprise / mobile / ICS ATT&CK (all published versions) + schema_design.md.
Documents/	Thai legal PDFs: Cybersecurity Act, PDPA, Criminal Code, Computer Crime Act B.E. 2550, Electronic Transactions Act B.E. 2544.
docker-compose.yml	Local orchestration: postgres, backend, rag-service, frontend.
.github/workflows/deploy.yml	CI/CD: build + push Docker images, trigger Railway redeploys.

Table 2-2: Top-level repository layout.

3. Technology Stack

3.1 Frontend

Component	Version	Purpose
Next.js	16.2.5	React framework (App Router, client components).
React / React DOM	19.2.4	UI runtime; React Compiler enabled via babel-plugin-react-compiler.
Tailwind CSS	4.2.4	Utility-first styling (@tailwindcss/postcss).
axios	1.16.0	HTTP client wrapped by src/lib/api.ts.
TypeScript	5.x	Static typing; strict interfaces for API responses.
Fonts	Manrope, JetBrains Mono	Loaded with next/font/google in layout.tsx.

Table 3-1: Frontend stack (from frontend/package.json).

3.2 Backend Gateway

Component	Purpose
FastAPI + Uvicorn	ASGI API server, automatic OpenAPI docs, lifespan hooks.
SQLAlchemy 2 (async) + asyncpg	Async ORM over PostgreSQL; DeclarativeBase models; session-per-request dependency.
Pydantic v2 + pydantic-settings	Request/response schemas and 12-factor configuration from env / .env.
httpx	Async proxy client to the RAG service (300 s timeout for long agent runs).
typhoon-ocr	Thai-capable OCR; converts uploaded PDF/images to Markdown before querying RAG.
Alembic	Database migration scaffolding (alembic.ini, versions/ currently empty).

Table 3-2: Backend stack (from backend/requirements.txt).

3.3 RAG / AI Stack

Component	Purpose
LangGraph	Stateful agent graph: routing, self-reflection loop, follow-up pause/resume.
LangChain (core, anthropic, ollama)	LLM bindings, message types, prompt templates, structured output.
FlagEmbedding (BGE-M3)	1024-dim dense + lexical sparse embeddings, FP16; one shared instance per process.
sentence-transformers (CrossEncoder)	BGE reranker scoring of (query, document) pairs.
qdrant-client	Hybrid search with named vectors and native RRF fusion (query_points + Prefetch).
neo4j (driver)	Cypher access for ingestion and subgraph expansion.

Component	Purpose
stix2 / json	Parsing MITRE ATT&CK STIX 2.1 bundles.
ragas, bert-score, rouge-score, datasets	Generation-quality evaluation (faithfulness, answer correctness, fallback metrics).

Table 3-3: RAG service stack (from `rag_service/requirements.txt`).

3.4 Models

Role	Cloud (default)	Local (--local mode)
Reasoning / translation / routing LLM	claude-sonnet-4-20250514	qwen2.5:7b (Ollama)
Evaluator / query-merger LLM	claude-haiku-4-5	gemma3:4b (Ollama)
RAGAS judge	claude-haiku-4-5 or qwen-2.5-72b (OpenRouter)	skipped (fallback metrics)
Embeddings	BAAI/bge-m3 (1024-dim, FP16)	same (runs locally)
Reranker	BAAI/bge-reranker-v2-m3	same
Fine-tuned specialist (optional)	—	mitre-qwen:7b (QLoRA, Ollama)

Table 3-4: Model assignments (`rag_service/app/RAG/GraphRAG/config.py`).

Settings that drive retrieval quality: `VECTOR_TOP_K=10`, `GRAPH_EXPANSION_DEPTH=2`, `FINAL_TOP_K=5`, RRF constant `RRF_K=60`, and `DUAL_QUERY_RETRIEVAL=true`. The evaluator is capped at 1024 output tokens; the main LLM at 4096.

4. Frontend (Next.js)

4.1 Pages

Route / File	Description
src/app/layout.tsx	Root layout. Registers Manrope (sans) and JetBrains Mono fonts, global metadata, globals.css.
src/app/page.tsx	Landing page. On mount calls getHealthStatus() and renders a live status pill (database connected / connecting / failed). Hero, feature cards, links to /chat.
src/app/chat/page.tsx	Chat interface. Message list with auto-scroll, loading dots, file attachment (PDF/PNG/JPG for OCR), suggested Thai starter questions, and full follow-up session handling.

Table 4-1: Frontend routes.

4.2 API Client (src/lib/api.ts)

All backend access goes through one typed client. `getApiBaseUrl()` reads `NEXT_PUBLIC_API_URL`, prepends `https://` when the protocol is missing, and appends `/api/v1` when absent. A build-time placeholder avoids breaking static builds; at runtime in the browser a missing URL throws.

Function	Endpoint	Purpose
<code>getHealthStatus()</code>	GET /health	Service + database status for the landing pill (5 s timeout).
<code>queryRag(query, useAgent)</code>	POST /rag/query	Main RAG query; <code>use_agent=true</code> selects the LangGraph agent.
<code>resumeRag(sessionId, answer)</code>	POST /rag/resume	Continues a paused follow-up session.
<code>queryRagFile(file, query)</code>	POST /rag/query-file	multipart/form-data upload; OCR then chain-mode RAG.
<code>chatContinue(query, history)</code>	POST /rag/query	Thin wrapper used by the chat page (history currently unused).

Table 4-2: API client functions.

4.3 Follow-Up Handling in the Chat UI

The chat page implements the client half of the follow-up protocol with a single piece of state, `currentSessionId`:

```
let response: QueryResponse;
if (currentSessionId) {
  response = await resumeRag(currentSessionId, currentInput);
} else if (currentFile) {
  response = await queryRagFile(currentFile, currentInput);
} else {
  response = await chatContinue(currentInput, messages);
}
if (response.status === "followup") {
  setCurrentSessionId(response.session_id || null); // next send resumes
  // render followup_question as the assistant message
```

```
} else {  
  setCurrentSessionId(null);           // session closed  
  // render response.answer  
}
```

Listing 4-1: chat/page.tsx handleSubmit() — routing a message to query / resume / file endpoints.

While a session id is set, the input placeholder switches to “Answer the follow-up question...” and the input border is highlighted amber, signaling that the next message answers the agent rather than starting a new query. Errors append a Thai apology message to the thread.

Note: the landing-page marketing copy mentions FAISS, BM25 and Claude 3 Haiku; the implemented stack actually uses Qdrant (dense+sparse RRF), BGE-M3 and Claude Sonnet/Haiku 4.x. The copy is legacy text, not a description of the pipeline.

5. Backend API Gateway (FastAPI, port 8000)

5.1 Application Setup (app/main.py)

The app is created with a lifespan context that verifies PostgreSQL connectivity at startup (SELECT 1) and disposes the engine pool at shutdown. Startup does not crash when the database is unreachable — the gateway still boots so the RAG proxy endpoints keep working. CORS is configured from settings (localhost:3000 always allowed; extra origins from the CORS_ORIGINS env var, auto-prefixed with both http/https when no protocol is given). Three routers are mounted under the /api/v1 prefix: health, users, rag.

5.2 Configuration (app/config.py)

A pydantic-settings `Settings` class reads environment variables (or backend/.env). The key property is `async_database_url`, which normalizes any provided `DATABASE_URL` to the `postgresql+asyncpg://` driver scheme (handling `postgres://` and `postgresql://` prefixes from managed providers like Railway), or constructs a URL from `POSTGRES_*` parts when `DATABASE_URL` is absent. `rag_service_url` defaults to `http://rag-service:8001` (the docker-compose service name).

5.3 Database Layer (app/database.py)

```
engine = create_async_engine(settings.async_database_url,
                             echo=settings.debug, pool_pre_ping=True)
async_session = async_sessionmaker(engine, class_=AsyncSession,
                                    expire_on_commit=False)

class Base(DeclarativeBase): pass

async def get_db():
    # FastAPI dependency
    async with async_session() as session:
        yield session
```

Listing 5-1: Async engine, session factory, ORM base and the get_db dependency.

5.4 REST Endpoints

Endpoint	Method	Behavior
/api/v1/health	GET	Returns {status, database, version}; database is connected/error based on a live SELECT 1.
/api/v1/users/	GET	Lists users (select(User) via async session).
/api/v1/users/	POST	Creates a user; 400 if the email already exists.
/api/v1/rag/query	POST	Proxies {query, use_agent} to RAG service /query; relays QueryResponse(status, answer, followup_question, session_id).
/api/v1/rag/query-file	POST	Multipart upload → Typhoon OCR → wraps extracted Markdown into a composed query → RAG /query with use_agent=false (chain mode).
/api/v1/rag/resume	POST	Proxies {session_id, answer} to RAG /resume to continue a paused agent session.
/api/v1/rag/generate-report	POST	Proxies to RAG /generate-report; validates the response against the CyberCaseReport schema.

Table 5-1: Gateway endpoints. All RAG routes are pure HTTP proxies with a 300 s timeout and error relaying (upstream status codes preserved).

5.5 Typhoon OCR Service (app/services/typhoon_ocr_reader.py)

This module turns an uploaded document into Markdown that can be injected into a RAG query:

- Validates extension (.pdf/.png/.jpg/.jpeg), non-empty content, and a 25 MB limit (413 above it).
- Persists the upload to a NamedTemporaryFile because typhoon_ocr operates on file paths; the temp file is always unlinked in a finally block.
- Resolves the API key from TYPHOON_OCR_API_KEY, TYPHOON_API_KEY, or OPENAI_API_KEY (the typhoon client uses the OpenAI-compatible env var, so the key is mirrored into OPENAI_API_KEY).
- Calls ocr_document(pdf_or_image_path, page_num) and 422s when OCR returns empty text.

The router then builds the final query: the user's question (or a default “analyze this document” instruction) followed by the extracted Markdown in a fenced block — `build_document_query()` — and sends it through chain mode (the linear pipeline), since document analysis does not need the follow-up loop.

5.6 Design Rationale

The gateway exists so that heavyweight ML dependencies (torch, FlagEmbedding, LangGraph) live only in the RAG service. The backend stays light (fast cold start, simple scaling), owns the relational data and uploads, and shields the frontend from RAG service topology. The two FastAPI apps define mirrored QueryRequest/QueryResponse models, so the proxy is a pass-through of the same JSON contract.

6. RAG Service (FastAPI, port 8001)

6.1 Startup: Shared Model Loading (app/main.py)

Loading BGE-M3 (~1.2 GB in FP16) and the reranker is expensive, so the lifespan hook loads each exactly once and injects the shared instances into every component:

```
embed_model = BGEM3FlagModel(EMBED_MODEL, use_fp16=USE_FP16)
reranker_model = Reranker(RERANKER_MODEL)

app.state.rag_chain = GraphRAGChain(embed_model=embed_model, reranker=reranker_model)
app.state.rag_agent = GraphRAGAgent(embed_model=embed_model, reranker=reranker_model)
app.state.retriever = HybridRetriever(embed_model=embed_model, reranker=reranker_model)
app.state.report_gen = ReportGenerator()
```

Listing 6-1: One embedding model and one reranker shared across chain, agent and standalone retriever. On any init failure the states are set to None and endpoints answer 503.

First boot is slow by design (model download/load + Neo4j/Qdrant connections). On shutdown each component's `close()` releases the Neo4j driver.

6.2 Endpoints

Endpoint	Method	Behavior
/health	GET	Reports whether rag_chain and rag_agent initialized.
/query	POST	use_agent=true → GraphRAGAgent.query() (may return status=followup + session_id). use_agent=false →

Endpoint	Method	Behavior
		GraphRAGChain.query() (always completed).
/resume	POST	GraphRAGAgent.resume(session_id, answer); 404 (KeyError) when the session is unknown or already consumed.
/generate-report	POST	Translates the query, runs dual-query hybrid retrieval, builds context, then ReportGenerator.generate() returns a CyberCaseReport.

Table 6-1: RAG service endpoints (no /api prefix — the gateway adds it).

7. GraphRAG Pipeline — Deep Dive

Everything in this chapter lives under `rag_service/app/RAG/GraphRAG/`.

Package	Files	Responsibility
<code>config.py</code>	—	Central settings: models, top-K, RRF weights, DB URLs, dual-query flag.
<code>models.py</code>	—	Pydantic models for ATT&CK entities and relationships.
<code>ingestion/</code>	<code>stix_parser</code> , <code>graph_loader</code> , <code>vector_loader</code>	STIX 2.1 → typed objects → Neo4j nodes/edges + Qdrant hybrid embeddings.
<code>retrieval/</code>	<code>vector_retriever</code> , <code>graph_retriever</code> , <code>reranker</code> , <code>hybrid_retriever</code>	Hybrid dense+sparse search, cross-encoder rerank, subgraph expansion, multi-query fusion.
<code>pipeline/</code>	<code>chain</code> , <code>agent_graph</code> , <code>router</code> , <code>cross_lingual</code> , <code>context_builder</code> , <code>evaluator</code> , <code>query_merger</code> , <code>report_generator</code>	Linear LCEL chain, LangGraph agent, and all LLM stages.
<code>evaluation/</code>	<code>eval_runner</code> , <code>retriever_metrics</code> , <code>generation_metrics</code> , <code>ground_truth</code> , <code>generate_eval_dataset</code> , <code>crosslingual_benchmark</code>	Offline quality benchmarks (chapter 12).
<code>main.py</code>	—	CLI entry point: <code>--ingest</code> / <code>--test</code> / <code>--agent</code> / <code>--retrieve-only</code> / <code>--local</code> .

Table 7-1: GraphRAG package map.

7.1 Typed Data Model (`models.py`)

All parsed STIX objects become Pydantic models. `AttackEntity` is the base (`stix_id`, `attack_id`, `name`, `description`, `node_label`, `url`, `domain`); subclasses add fields: `Technique` (`platforms`, `is_subtechnique`, `tactics`), `Group/Software/Campaign` (`aliases`; `software` also has `software_type` `tool|malware`), `Tactic` (shortcode such as `initial-access`), `DataSource` and `DataComponent`. `AttackRelationship` mirrors a STIX relationship (`relationship_type`, `source_ref/target_ref`, `source/target` names for embedding, `description`, `edge_label`).

7.2 Ingestion Subsystem

7.2.1 StixParser (`ingestion/stix_parser.py`)

Parses every *.json bundle in the configured domain folders (enterprise + mobile by default) in two passes:

- **Pass 1 — entities.** Each STIX object type maps to a node label (`attack-pattern` → `Technique/Subtechnique` via `x_mitre_is_subtechnique`, `intrusion-set` → `Group`, `tool|malware` → `Software`, `campaign` → `Campaign`, `course-of-action` → `Mitigation`, `x-mitre-tactic` → `Tactic`, `x-mitre-data-source/component` → `DataSource/DataComponent`). Revoked or deprecated objects are skipped. Lookup tables (`id` → `name`, `id` → `label`, `tactic shortcode` → `id`, `data-component` → `source`) are built during this pass.

- **Pass 2 — relationships.** STIX relationship objects are mapped through RELATIONSHIP_TYPE_MAP (uses → USES, mitigates → MITIGATES, subtechnique-of → SUBTECHNIQUE_OF, attributed-to → ATTRIBUTED_TO, detects → DETECTS); a relationship is kept only when both endpoints were parsed (dangling references dropped).
- **Derived edges.** IN_TACTIC edges are synthesized from each technique's kill_chain_phases via the tactic shortname lookup; HAS_COMPONENT edges from x_mitre_data_source_ref. Their stix_id is a synthetic 'derived--...' key.
- **Deduplication.** parse_all_domains() collapses entities and relationships by stix_id, because the data folder holds every published ATT&CK version of the same bundle.

7.2.2 GraphLoader (ingestion/graph_loader.py)

Full reload semantics: load_all() runs clear → constraints → indexes → nodes → edges.

```
// nodes: batched UNWIND-MERGE per label; every node also gets :Entity
UNWIND $batch AS props
MERGE (n:Technique {stix_id: props.stix_id})
SET n:Entity, n += props

// edges: O(1) endpoint match via the :Entity uniqueness constraint
UNWIND $rels AS rel
MATCH (src:Entity {stix_id: rel.source_ref})
MATCH (tgt:Entity {stix_id: rel.target_ref})
CREATE (src)-[r:USES {stix_id: rel.rel_id, description: rel.description}]->(tgt)
```

Listing 7-1: Bulk loading pattern (batch size 5,000). The shared :Entity label + uniqueness constraint lets edge creation match endpoints without scanning per-label indexes; CREATE (not MERGE) is safe because relationships were deduplicated in Python and the DB was cleared.

Uniqueness constraints exist on stix_id for every label (Technique, Subtechnique, Group, Software, Campaign, Mitigation, Tactic, DataSource, DataComponent, Entity); secondary indexes cover Technique/Subtechnique.attack_id, Group.name, Software.name, Tactic.shortname. Long descriptions are truncated to 5,000 characters.

7.2.3 VectorLoader (ingestion/vector_loader.py)

Recreates both Qdrant collections with named vectors — dense (1024-dim cosine) and sparse — then embeds in batches of 16:

- **Entity documents:** "{node_label}: {name}. {description}" (truncated to 8,000 chars); skipped when the description is empty.
- **Relationship documents:** "{source} {EDGE_LABEL} {target}: {description}" — relationship descriptions often contain the most incident-specific text in ATT&CK (e.g., exactly how APT29 conducts phishing).
- **Point IDs:** Qdrant requires UUIDs, so uuid_from_stix_id() reuses the UUID suffix of the STIX id when valid, otherwise derives one from an MD5 hash of the full id.
- **Payload:** the full metadata (stix_id, attack_id, entity_type Node|Relationship, node/edge label, names, domain, url) plus the embedded text under document — this is what retrieval returns.

7.3 Retrieval Subsystem

7.3.1 VectorRetriever — Qdrant hybrid search

Each query is encoded once by BGE-M3 into a dense vector and a sparse lexical-weight dict; Qdrant then fuses two prefetch branches with native Reciprocal Rank Fusion:

```
results = client.query_points(
    collection_name=collection_name,
```

```

prefetch=[
    Prefetch(query=dense_vec, using="dense", limit=max(top_k*5, 50)),
    Prefetch(query=SparseVector(indices=..., values=...),
              using="sparse", limit=max(top_k*5, 50)),
],
query=FusionQuery(fusion=Fusion.RRF),
limit=top_k, with_payload=True)

```

Listing 7-2: `_search_hybrid()`. Dense catches semantics (incl. Thai → English cross-lingual matching); sparse catches exact tokens such as 'T1566' or 'Mimikatz'. Each branch over-fetches 5× before fusion.

`search_all()` queries both collections — entities get the full top-K quota, relationships half ($\max(\text{top_k}/2, 3)$) biasing toward Technique/Tactic nodes — then min-max normalizes scores within each collection (RRF scores from different collections are not directly comparable) and merges to the top-K.

7.3.2 Reranker (retrieval/reranker.py)

A cross-encoder (BAAI/bge-reranker-v2-m3, multilingual including Thai) jointly scores each (query, document[:512]) pair, replacing the fusion score with `sigmoid(logit)` so scores stay in [0,1] for display. Results are re-sorted before the top-K cut that feeds graph expansion and the LLM context. The model choice matters for dual-query mode: the previous mmarco reranker was not trained on Thai and mis-scored Thai ↔ English pairs.

7.3.3 GraphRetriever — Neo4j subgraph expansion

Given the STIX ids that survived reranking, `expand()` fetches for each id the center node plus all outgoing and incoming relationships (one hop each way) into a `SubgraphResult` (center_node, neighbors, edges). `to_text()` renders the subgraph for the LLM, grouping edges by type with display names (USES → Used by, MITIGATES → Mitigated by, IN_TACTIC → Belongs to tactic, SUBTECHNIQUE_OF → Parent technique, DETECTS → Detected by) and attaching relationship-description previews. A utility `get_multi_hop_path()` answers 'how are X and Y related' via `shortestPath` up to 4 hops.

7.3.4 HybridRetriever — orchestration and multi-query fusion

`retrieve()` runs the chain: vector search → rerank → seed-id extraction → graph expansion. Seed extraction walks reranked results in order: Node results contribute their own `stix_id`; Relationship results contribute their source and target ids; collection stops at `FINAL_TOP_K` (5) seeds, so graph expansion follows reranker relevance.

`retrieve_multi(queries)` is what the pipeline actually calls. It runs `retrieve()` per query (English translation, original Thai, plus any follow-up rewrites), then merges:

- Vector results dedup by `stix_id` keeping the highest score, then re-sorted descending.
- Graph subgraphs dedup by center-node `stix_id` keeping the first (identical for the same seed).

The merged `GraphRAGResult(vector_results, graph_results)` is handed to the context builder.

7.4 Context Builder (pipeline/context_builder.py)

`build_context()` assembles the LLM context: a header, then the top-5 semantic matches (entity type, label, name, ATT&CK id, relevance score, first 600 chars of the document), then up to 3 graph subgraphs rendered by `to_text()`; total capped at 10,000 characters. `build_generation_prompt()` wraps that context with the user question (original + English translation when they differ) and instructions. When follow-up answers exist, a **CONFIRMED INCIDENT FACTS** block is injected before the context and marked as ground truth the model must incorporate — this is how user-provided clarifications dominate generation.

7.5 Cross-Lingual Layer (pipeline/cross_lingual.py)

Three LLM stages with strict separation of concerns:

Stage	Prompt contract	Output
1. Query translation	Translate Thai → English for searching ATT&CK; preserve technical terms and ATT&CK IDs (T####, G####, S####, M####, TA####, C####); return query unchanged if already English.	English retrieval query
2. Reasoning	Rewrite retrieved incident context into plain English for prosecutors/law enforcement: simplify jargon via a built-in glossary (e.g., privilege escalation → gain administrator-level control), preserve event order and identifiers, no fabrication, no legal interpretation. Output exactly four sections: INCIDENT SUMMARY, ATTACK SEQUENCE, MITRE ATT&CK TECHNIQUES IDENTIFIED, IMPACT ASSESSMENT.	Simplified English narrative
3. Thai translation	Translate the narrative to Thai for non-technical readers; keep ATT&CK IDs, technique/group/software names, CVE ids in English; translate the four section headers to fixed Thai equivalents; add nothing.	Thai answer (only when the query was Thai)

Table 7-2: The three LLM stages. Language detection is a Thai Unicode range regex; `should_respond_in_thai()` keys the final stage.

Dual-query retrieval

Translate-then-retrieve alone makes the translation a single point of failure: one bad translation poisons dense, sparse, and every follow-up rewrite. `build_retrieval_queries()` therefore returns `[english_query]` and, when `DUAL_QUERY_RETRIEVAL` is on and the original is Thai (and differs from the translation), appends the raw Thai query as a second retrieval channel — BGE-M3's cross-lingual dense space plus any English keywords embedded in the Thai text (sparse) can recover what the translation missed. Follow-up rewrites are appended after these two; duplicates are filtered.

7.6 Query Router (pipeline/router.py)

A small LLM call classifies each query as `GENERAL_EXPLANATION` (concept/definition questions — answered directly from model knowledge, no retrieval) or `INCIDENT_ANALYSIS` (attacker actions/forensic narratives — full RAG pipeline). The system prompt strictly limits the model to returning one label (`max_tokens=32`) and the parser defaults to `INCIDENT_ANALYSIS`. **Note:** the LCEL chain honors the router, but in the agent the routing edge is currently hard-coded to the incident path (the `GENERAL_EXPLANATION` branch is commented out in `_edge_after_route`) — see chapter 15.

7.7 Linear Pipeline — GraphRAGChain (pipeline/chain.py)

The non-agentic mode used by `/query` with `use_agent=false` and by the OCR file endpoint. Sequence: `route` → (general-explanation shortcut) → `detect language + translate` → `retrieve_multi(build_retrieval_queries(...))` → `build_context` → reasoning LLM (English narrative)

→ translation LLM (Thai only when needed). With no LLM configured it degrades gracefully and returns the raw retrieved context. Verbose mode prints the top sources with scores.

7.8 Agentic Pipeline — GraphRAGAgent (pipeline/agent_graph.py)

The agent wraps the same retrieval core in a LangGraph state machine that adds self-reflection, follow-up questioning, and fallback strategies.

7.8.1 State and response types

AgentState field group	Fields
Inputs / routing	original_query, verbose, route
Translation	english_query, respond_in_thai
Retrieval	graphrag_result, context, rewritten_queries (all MITRE-aligned rewrites)
Evaluation	evaluation (EvaluationResult), retry_count
Follow-up	followup_question, followup_answer, awaiting_followup, followup_count, broaden_count, incident_facts {slot: value}, asked_slots {slot,...}
Fallback / output	strategy, gap_warning, acknowledgement_message, answer

Table 7-3: AgentState (TypedDict, total=False) — the single mutable object flowing through all nodes.

AgentResponse is the public result: status is either completed (answer filled) or followup (followup_question + session_id filled). to_dict() serializes it for the API.

7.8.2 Graph topology

```

route_query --(general)--> general_explanation --> END
|
(incident)
v
translate_query --> retrieve --> evaluate_context
|
| sufficient | insufficient | broaden
|   v       |   v         |   |
| reasoning | prepare_followup -> END (pause)
|   |       |
| (respond_in_thai?)
|   v
| translate_output --> END
+---- broaden_search (loops back to retrieve)

```

Figure 7-1: Compiled LangGraph. Conditional edges: _edge_after_route, _edge_after_evaluation (sufficient | followup | broaden), _edge_after_reasoning (translate | done).

7.8.3 Node behavior

- `route_query` — calls QueryRouter (result currently overridden to incident at the edge).
- `general_explanation` — direct LLM answer, optionally in Thai; no retrieval.
- `translate_query` — sets english_query and respond_in_thai.
- `retrieve` — builds the full query list (English + Thai original + accumulated rewrites) and calls retrieve_multi(top_k=10); stores assembled context.
- `evaluate_context` — calls ContextEvaluator with retry_count = followup_count + broaden_count, plus incident_facts and asked_slots so it never re-asks answered slots (chapter 8).

- `prepare_followup` — sets `awaiting_followup=true` and copies the evaluator's `follow_up` question; graph then reaches END in the paused state.
- `broaden_search` — appends the evaluator's `new_query` to `rewritten_queries`, increments `broaden_count`, loops back to retrieve (hard cap: 2 broaden loops).
- `reasoning` — builds the generation prompt (context + confirmed facts) and invokes the reasoning LLM. Fast path: if strategy is `ACKNOWLEDGE_LIMIT`, the message is returned directly, but only when total retries ≥ 2 — guarding against local models that emit the strategy prematurely.
- `translate_output` — Thai rendering of the English answer when `respond_in_thai`.

7.8.4 Decision logic after evaluation

```

if evaluation is None or verdict == SUFFICIENT: return "sufficient"
total = followup_count + broaden_count
if total >= MAX_FOLLOWUP_RETRIES: # = 2
    if strategy == "BROADEN_SEARCH" and broaden_count < 2: return "broaden"
    return "sufficient" # PARTIAL ANSWER / ACKNOWLEDGE_LIMIT fall through
return "followup" # INSUFFICIENT and retries remain

```

Listing 7-3: `_edge_after_evaluation` — bounded self-reflection: at most 2 user follow-ups and 2 broaden loops before the agent answers with what it has.

7.8.5 Dual-mode `query()`: CLI callback vs API sessions

`query()` seeds the initial state and invokes the graph. If the run ends awaiting a follow-up:

- **CLI mode** — a `followup_callback(question)` is called synchronously; the loop continues through multiple follow-up rounds until the graph completes. An empty answer triggers `_force_continue()`, which clears the evaluation and re-invokes so reasoning proceeds with existing context.
- **API mode** — the full graph state is parked in an in-memory dict under a uuid4 session id, and `AgentResponse(status=followup)` is returned. `resume(session_id, answer)` pops the state (one-time use; unknown id $\rightarrow$ `KeyError` $\rightarrow$ HTTP 404) and continues.

`Resumption(_resume_with_answer)` performs five steps: (1) store the answer in `incident_facts` under the evaluator's `missing_slot`; (2) `QueryMerger` produces a MITRE-aligned rewritten query; (3) the rewrite is appended to `rewritten_queries` (the original `english_query` is never mutated); (4) the slot is recorded in `asked_slots`; (5) `followup/retry` counters are incremented and the graph is re-invoked from the start — retrieve then naturally runs all accumulated queries in parallel.

8. Module: Context Evaluator — Self-Reflection (`pipeline/evaluator.py`)

The evaluator is the component that makes the pipeline agentic. After every retrieval it judges whether the assembled context can answer the query, using the lightweight `claude-haiku-4-5` (or `gemma3:4b` locally) with temperature 0 and a 1,024-token budget.

8.1 Verdicts and Result Shape

The LLM must return JSON parsed into `EvaluationResult`: `verdict` (`SUFFICIENT` | `INSUFFICIENT`), `reason`, `covered_phases` / `missing_phases`, `missing_slot`, `follow_up`, `strategy`, `new_query`, `gap_warning`, `message`. A legacy `NEED_CLARIFICATION` constant remains for backwards compatibility but is no longer produced.

8.2 Judging Rules (system prompt)

- Judge semantic coverage, not keyword match — a technique counts as covered when its description maps to the behavior even if not named.
- Check four attack phases: Initial Access, Credential Access, Privilege Escalation, Impact/Data Destruction.
- Thresholds: ≥ 2 phases covered → SUFFICIENT; 1 phase clearly explained → SUFFICIENT; otherwise INSUFFICIENT with a follow-up about the most critical missing phase. Single-technique incidents (only SQL injection, only phishing) must not be marked INSUFFICIENT merely because later phases are absent.
- Prefer SUFFICIENT when in doubt — better to answer with partial context than to over-ask.

8.3 Slot Tracking (multi-turn interrogation)

The evaluator maintains an ordered slot model of an incident: `initial_access` → `credential_theft` → `privilege_escalation` → `lateral_movement` → `impact`. Python (not the LLM) computes the next missing slot — the first slot present neither in `incident_facts` nor `asked_slots` — and injects it into the prompt together with the filled slots. The prompt also pins common confusions (phishing is `credential_theft` or `initial_access`, never `privilege_escalation`; logging in with a stolen admin password is Valid Accounts T1078, not a privilege-escalation exploit). Known facts are echoed under a KNOWN INCIDENT FACTS header with an instruction never to re-ask, and the follow-up question must target only the next missing slot, in the user's language.

8.4 Fallback Strategies (retries exhausted)

Strategy	When	Effect in the agent
BROADEN_SEARCH	The missing item is a sub-technique-level detail.	Evaluator supplies <code>new_query</code> (parent technique + broader tactic); agent loops retrieval with it (max 2 broaden loops).
PARTIAL_ANSWER	≥ 2 phases covered, one detail missing.	Answer proceeds; <code>gap_warning</code> flags the gap.
ACKNOWLEDGE_LIMIT	< 2 phases covered even after retries.	Reasoning short-circuits and returns the evaluator's message stating the knowledge base lacks data for this pattern.

Table 8-1: Fallback strategies. *FORCE_SUFFICIENT* is explicitly forbidden — answering from known-incomplete context without flagging gaps would mislead the analyst.

8.5 Safety Valves

- **Max-retry guard:** at `retry_count` ≥ 2 the evaluator returns SUFFICIENT without calling the LLM (no infinite loops, no wasted tokens).
- **Retry hint:** on iterations ≥ 1 the prompt asks for looser criteria — prefer SUFFICIENT when context is at least partially on-topic.
- **Three-stage JSON parsing:** (1) regex for the first `{...}` block (handles markdown fences); (2) outermost brace-pair scan (handles truncated output); (3) default to SUFFICIENT — the pipeline is never blocked by a malformed judge response.
- **Context cap:** only the first 4,000 characters of context are sent to the judge.

9. Module: Follow-Up (interactive clarification)

The follow-up module spans four files: the evaluator decides that a question is needed and writes it; agent_graph pauses/parks/resumes the session; query_merger turns the user's reply into a retrieval-ready rewrite; the frontend chat page carries the session id.

9.1 Full Sequence

```

1 user: POST /rag/query {query}
2 agent: retrieve -> evaluate = INSUFFICIENT (missing_slot=initial_access)
3 agent: prepare_followup -> graph END (awaiting_followup)
4 service: park state under session_id = uuid4()
5 api: {status: followup, followup_question, session_id}
6 user: POST /rag/resume {session_id, answer}
7 agent: incident_facts[initial_access] = answer
8 agent: QueryMerger.merge() -> MITRE-aligned rewritten query
9 agent: re-invoke graph: retrieve_multi(english + thai + rewrites)
10 agent: evaluate again (looser) -> reasoning -> answer
11 api: {status: completed, answer}

```

Figure 9-1: One follow-up round over the API. At most two rounds occur (MAX_FOLLOWUP_RETRIES=2).

9.2 QueryMerger (pipeline/query_merger.py)

Naive concatenation of question + answer produces bloated queries that confuse the retriever. QueryMerger makes one haiku-class LLM call that merges the original query, the clarifying question, and the user's answer into a 1–4 sentence **self-contained English retrieval query** with MITRE technique keywords placed first (e.g., an answer of 'SQL Injection' becomes 'Exploit Public-Facing Application SQL Injection T1190 ...'), preserving the rest of the incident chain. Without an LLM it degrades to simple concatenation.

9.3 Properties Worth Noting

- Sessions are an in-process dict (GraphRAGAgent._sessions) — one-time-use, popped on resume; a restart or a second replica loses them (chapter 15).
- Answers become structured ground truth: incident_facts are injected into both the evaluator prompt (do not re-ask) and the generation prompt (CONFIRMED INCIDENT FACTS — must be reflected in the answer).
- Original queries are never mutated; every rewrite is additive, and retrieve_multi merges all channels by best score, so a misleading clarification cannot erase earlier evidence.
- rag_service/app/test_agent_flow.py exercises the loop with intentionally vague Thai incidents and scripted callback answers.

10. Module: Report Generation (pipeline/report_generator.py)

POST /generate-report produces a formal, structured incident report rather than a chat answer. The endpoint translates the query, runs the same dual-query hybrid retrieval, builds context, and calls ReportGenerator.

10.1 CyberCaseReport Schema (7 sections)

Field	Type	Content
case_summary	str	5.1 Concise incident summary, written in Thai.
detected_indicators	list[str]	5.2 IoCs / artifacts: hashes, IPs, files.
mitre_mapping	list[str]	5.3 Mapped ATT&CK techniques (e.g., T1566).
mapping_justification	str	5.4 Why those techniques were chosen.
evidence_to_investigate	list[str]	5.5 Logs / data sources analysts should check.
preliminary_recommendations	list[str]	5.6 Immediate mitigation / remediation actions.
system_limitations	str	5.7 Caveats and missing data limiting accuracy.

Table 10-1: CyberCaseReport (Pydantic). The identical schema is duplicated in backend/app/schemas/rag.py so the gateway re-validates the proxied response.

10.2 Schema-Enforced Generation

```
self.structured_llm = self.llm.with_structured_output(CyberCaseReport)
chain = self.prompt | self.structured_llm
return chain.invoke({"query": query, "context": context})
```

Listing 10-1: with_structured_output() makes Claude return the exact Pydantic shape (tool-call based), so the API never returns malformed reports. Temperature is 0; the system prompt casts the model as a CTI analyst writing in Thai, grounded only in the provided context, pushing unknowns into system_limitations.

11. Database Schemas

11.1 PostgreSQL (application data)

One table exists today — users — defined as an async SQLAlchemy model (backend/app/models/user.py) and validated by UserCreate/UserRead schemas:

Column	Type	Constraints
id	UUID	PK, default uuid4
email	VARCHAR(255)	UNIQUE, NOT NULL, indexed
display_name	VARCHAR(100)	NULL allowed
created_at	TIMESTAMPTZ	server_default now()
updated_at	TIMESTAMPTZ	server_default now(), onupdate now()

Table 11-1: users table. Alembic is scaffolded (alembic.ini, env template) but backend/alembic/versions/ holds no migrations yet — the table must be created by a future migration or manual DDL.

11.2 Neo4j (MITRE ATT&CK knowledge graph)

The graph follows the design in Mitre_ATT&CK Doc/schema_design.md. Every node also carries the shared :Entity label with a uniqueness constraint on stix_id.

Node label	STIX type	Key properties
Technique	attack-pattern	stix_id, attack_id (T1566), name, description, platforms, tactics, url, domain
Subtechnique	attack-pattern	same + is_subtechnique=true (T1566.001)
Group	intrusion-set	attack_id (G0016), aliases
Software	tool / malware	attack_id (S0154), aliases, software_type
Campaign	campaign	attack_id (C0015), aliases
Mitigation	course-of-action	attack_id (M1036)
Tactic	x-mitre-tactic	attack_id (TA0001), shortname (initial-access)
DataSource	x-mitre-data-source	attack_id, platforms
DataComponent	x-mitre-data-component	name, description

Table 11-2: Node labels.

Edge label	Source → Target	Origin
USES	Group/Software/Campaign → Technique/Software	STIX uses (carries description — key RAG context)
MITIGATES	Mitigation → Technique	STIX mitigates (carries description)
SUBTECHNIQUE_OF	Subtechnique → Technique	STIX subtechnique-of
ATTRIBUTED_TO	Campaign → Group	STIX attributed-to
DETECTS	DataComponent → Technique	STIX detects
IN_TACTIC	Technique → Tactic	derived from kill_chain_phases

Edge label	Source → Target	Origin
HAS_COMPONENT	DataSource → DataComponent	derived from x_mitre_data_source_ref

Table 11-3: Edge labels. All edges store stix_id and description (truncated to 5,000 chars).

11.3 Qdrant (vector store)

Two collections, both using named vectors — dense (1024-dim, cosine) and sparse (lexical weights) — enabling server-side hybrid search. Ingestion-time counts observed in evaluation logs: **2,733** entity documents and **25,467** relationship documents.

```
// mitre_entities payload
{ "stix_id": "intrusion-set--899ce53f-...", "attack_id": "G0016",
  "entity_type": "Node", "node_label": "Group", "name": "APT29",
  "domain": "enterprise", "url": "https://attack.mitre.org/groups/G0016",
  "document": "Group: APT29. APT29 is a threat group attributed to ..." }

// mitre_relationships payload
{ "stix_id": "relationship--008ef61a-...", "entity_type": "Relationship",
  "edge_label": "USES", "source_id": "intrusion-set--...",
  "target_id": "attack-pattern--...", "source_name": "APT29",
  "target_name": "WMI", "document": "APT29 USES WMI: APT29 has used ..." }
```

Listing 11-1: Payload schemas. source_id/target_id let the hybrid retriever seed graph expansion from relationship hits; node_label/edge_label support filtered search.

12. Module: Evaluation (offline quality benchmarks)

Located at `rag_service/app/RAG/GraphRAG/evaluation/`. The CLI orchestrator is `eval_runner.py`:

```
cd rag_service/app/RAG/GraphRAG
python -m evaluation.eval_runner --dataset evaluation/eval_dataset.json --mode retriever
python -m evaluation.eval_runner --dataset ... --mode generation
python -m evaluation.eval_runner --dataset ... --mode full
# options: --local (Ollama), --output results.md, --max-samples N
```

Listing 12-1: Evaluation entry points. Samples with more than 50 relevant STIX ids are filtered out as unanswerable list questions.

12.1 Retriever Benchmarks (retriever_metrics.py)

Three retriever configurations are benchmarked side by side — Vector-only (Qdrant), Graph-only (Neo4j keyword/ATT&CK-id Cypher search + 1-hop expansion), and Hybrid (vector + rerank + graph expansion, collecting both vector ids and subgraph node ids). Metrics, computed at $K \in \{1, 3, 5, 10\}$ with mean aggregation and latency tracking:

Metric	Definition
Hit@K	1 if any relevant document appears in the top-K.
Recall@K	Fraction of all relevant documents found in the top-K.
Precision@K	Fraction of the top-K that is relevant.
MRR	Mean reciprocal rank of the first relevant result.
NDCG@K	Rank-discounted gain normalized by the ideal ranking (binary relevance).
MAP	Mean of average precision at each relevant position.

Table 12-1: Retriever metrics (pure functions over ordered STIX-id lists vs. ground-truth sets).

12.2 Generation Benchmarks (generation_metrics.py)

- **RAGAS metrics** (when installed): faithfulness — is the answer grounded in retrieved context; answer_correctness — vs. the reference answer (added only when references exist). Judge priority: Claude Haiku → OpenRouter qwen-2.5-72b; embeddings for the semantic component come from local Ollama nomic-embed-text to avoid OpenAI quota; RunConfig throttles to 2 workers with long backoff to survive rate limits. answer_relevancy is disabled because RAGAS would require OpenAI embeddings for it.
- **Fallback metrics** (always available, stdlib-only): Token F1 (set precision/recall over tokens), ROUGE-L (LCS-based F1), plus BERTScore when installed. Used alone in fully local mode, where 4B–7B judges cannot emit the structured NLI output RAGAS needs.

12.3 Ground Truth and Dataset Generation

Each EvalSample is {query, relevant_stix_ids, reference_answer, language, category}. generate_eval_dataset.py builds datasets from Neo4j itself — “the graph IS the ground truth”: template questions are instantiated over well-connected nodes and every relevant_stix_ids list is produced by the same Cypher query the knowledge base would answer, eliminating manual under-labeling, contradictions and empty ground truth. A validator rejects samples with zero ids.

Dataset	Samples	Notes
eval_dataset.json	136	Curated English set; 10 categories (technique_lookup, mitigation_lookup, group_techniques, ...).
eval_dataset_generated.json	2,193	Neo4j-generated, validated.
Thai_dataset.json	2,223	Thai queries incl. incident_analysis category — drives cross-lingual eval.
Thai_dataset_08.json	3,324	Extended Thai set.

Table 12-2: Evaluation datasets shipped in the repository.

12.4 Cross-Lingual Benchmark (crosslingual_benchmark.py)

Compares three retrieval configurations for Thai queries against the English-only knowledge base: **tRAG** (translate-then-retrieve only), **Thai-direct** (raw Thai query, relying on BGE-M3's cross-lingual dense space), and **Dual-query** (both in parallel, merged by max score — the production setting). Translations are cached in translation_cache.json so re-runs are free and all configurations see identical translations; reported latency excludes the translation call. This benchmark is the empirical basis for enabling DUAL_QUERY_RETRIEVAL (see docs/DUAL_QUERY_UPGRADE.md).

13. Module: Fine-Tune — MITRE ATT&CK Specialist (rag_service/finetune/)

Goal: turn the local generation model (qwen2.5:7b via Ollama) into a MITRE specialist while keeping the base model untouched for honest A/B comparison. The fine-tuned model registers under a separate Ollama name, mitre-qwen:7b; switching the whole pipeline between models is done purely via the `LOCAL_LLM_MODEL` env var — zero pipeline code changes. Training runs on free cloud GPUs (Colab/Kaggle T4 16 GB) because the dev machine's RTX 2050 (4 GB) cannot fit QLoRA for a 7B model; inference of the resulting Q4 GGUF runs locally.

13.1 Workflow

```
1 python data/build_dataset.py --max-per-category 600 (local)
   STIX bundles -> StixParser -> Q&A pairs -> train.jsonl / val.jsonl
2 python train/train_unsloth.py --gguf (cloud GPU)
   Qwen2.5-7B-Instruct 4-bit + LoRA -> adapter -> GGUF Q4_K_M
3 ollama create mitre-qwen:7b -f export/Modelfile (local)
4 python compare/run_comparison.py --max-samples 20 (local)
   eval_runner twice (base vs ft) -> side-by-side metric table
```

Figure 13-1: Four-step fine-tune lifecycle.

13.2 Dataset Builder (data/build_dataset.py + templates.py)

- Reuses the production StixParser; by default parses only the newest STIX version per domain (enterprise + mobile).
- Two example flavours: closed-book (Q → A, baking ATT&CK knowledge into weights) and grounded (context + Q → A, mirroring the RAG inference shape) — current build: 3,631 closed / 270 grounded.
- templates.py generates per-category Q&A wording that deliberately mirrors the eval reference_answer style; clean_text() strips MITRE citation/link markup and never truncates mid-word (sentence- then word-boundary), because an earlier dataset version with mid-word cuts taught the model to garble.
- English-only output by design — Thai translation is a separate downstream pipeline stage.
- Holdout control: --holdout ids removes entities appearing in eval_dataset.json for leak-free testing; default none trains on the full KB (interpreted as in-domain knowledge absorption).

Current dataset_stats.json: **3,901 examples** (3,589 train / 312 val ≈ 8% split), 10 categories — technique_lookup 600, technique_profile 600, mitigation_lookup 600, software_techniques 600, software_type_query 600, technique_groups 515, group_techniques 173, group_software 161, tactic_techniques 27, campaign_attribution 25.

13.3 Training Configuration (ft_config.py, train_unsloth.py)

Parameter	Value
Base checkpoint	Qwen/Qwen2.5-7B-Instruct, loaded 4-bit via Unsloth (same family/size as the Ollama base for apples-to-apples A/B)
LoRA	r=16, alpha=16, dropout=0, target modules: q/k/v/o_proj + gate/up/down_proj
Schedule	2 epochs, lr 2e-4 cosine, warmup 3%, weight decay 0.01
Batch	per-device 2 × grad-accum 8 = effective 16; max sequence 2,048
Loss masking	train_on_responses_only — loss on assistant tokens only

Parameter	Value
Export	LoRA adapter → merge → GGUF Q4_K_M (Unsloth save_pretrained_gguf, or merge_and_gguf.py via transformers + llama.cpp as fallback) → Ollama Modelfile

Table 13-1: QLoRA hyperparameters.

13.4 A/B Comparison (compare/run_comparison.py)

Runs the existing generation evaluation twice as subprocesses, switching only the LOCAL_LLM_MODEL env var (base → fine-tuned), parses both reports, and prints a side-by-side table with deltas across Faithfulness, Answer Relevancy, Answer Correctness, Token F1, ROUGE-L, BERTScore F1 and latency. Results for past runs are committed under compare/results_*.md.

14. Deployment & Operations

14.1 Local Orchestration (docker-compose.yml)

Four services: postgres:16-alpine (host port 5433, healthcheck-gated), backend (depends on db healthy + rag-service started, hot-reload via volume mount), rag-service (HF_HUB_OFFLINE=1 — models are baked into the image), and frontend (NEXT_PUBLIC_API_URL passed as build arg and runtime env). Neo4j and Qdrant are cloud-hosted; no local containers. Run with `doppler run -- docker compose up --build`.

14.2 Secrets & Environment Variables

Variable	Consumer	Purpose
ANTHROPIC_API_KEY	backend, rag-service	Claude LLM calls (required).
DATABASE_URL / POSTGRES_*	backend	PostgreSQL connection (async URL normalized in config).
RAG_SERVICE_URL	backend	Proxy target (default http://rag-service:8001).
TYPHOON_OCR_API_KEY / TYPHOON_API_KEY	backend	Typhoon OCR; mirrored into OPENAI_API_KEY for the client.
NEO4J_URI / NEO4J_USER / NEO4J_PASSWORD	rag-service	Knowledge graph (Aura: neo4j+s://...).
QDRANT_URL / QDRANT_API_KEY	rag-service	Vector store (cloud). QDRANT_HOST/PORT for local; in-memory fallback otherwise.
OPENROUTER_API_KEY	rag-service	Optional RAGAS judge (qwen-2.5-72b).
DUAL_QUERY_RETRIEVAL	rag-service	Toggle Thai+English dual retrieval (default true).
OLLAMA_BASE_URL / LOCAL_LLM_MODEL / LOCAL_EVAL_MODEL	rag-service	Local --local mode model selection.
NEXT_PUBLIC_API_URL	frontend	Backend base URL (build-time + runtime).
CORS_ORIGINS	backend	Extra allowed origins (comma-separated).

Table 14-1: Environment variables. Doppler injects secrets in all environments; local .env files remain supported.

14.3 CI/CD (GitHub Actions → Docker Hub → Railway)

deploy.yml triggers on push to main and manual dispatch. Two parallel jobs build and push kkhw/cybercase-framework-backend:latest and ...-frontend:latest. On manual dispatch (with per-service booleans) the workflow calls Railway's GraphQL API: it queries the service's latest deployment id, then issues a deploymentRedeploy mutation — effectively 'pull latest image and restart'. The frontend build injects NEXT_PUBLIC_API_URL as a build arg from repo secrets.

14.4 Local Development Commands

```
python install_deps.py          # installs backend + rag service deps
cd backend && uvicorn app.main:app --reload    # :8000
cd rag_service && uvicorn app.main:app --port 8001 --reload
cd rag_service/app && python -m RAG.GraphRAG.main --ingest # load KB
cd rag_service/app && python -m RAG.GraphRAG.main --agent  # CLI agent
cd frontend && npm run dev                                # :3000
```

Listing 14-1: Common commands. The CLI must run as a module (relative imports); main.py also patches Windows consoles to UTF-8 for Thai output.

15. Known Issues & Implementation Notes

Item	Detail
STIX path mismatch	config.py derives <code>_PROJECT_ROOT</code> three parents up from GraphRAG/ — which resolves to <code>rag_service/</code> — so <code>ATTACK_DOMAINS</code> points at <code>rag_service/Mitre_ATT&CK Doc</code> , a path that does not exist; the bundles live at the repo root. <code>--ingest</code> fails until the path is fixed; <code>ft_config.py</code> already works around it by resolving the real location itself.
Router bypassed in agent	<code>_edge_after_route</code> always returns 'incident' (GENERAL_EXPLANATION branch commented out), so the agent retrieves even for concept questions. The LCEL chain still honors the router.
In-memory sessions	Follow-up sessions live in <code>GraphRAGAgent._sessions</code> (a dict). A service restart or horizontal scaling loses/strands sessions; no TTL eviction exists. A shared store (e.g., Redis) would be required for multi-replica deployments.
No migrations	<code>alembic/versions/</code> is empty; the users table has no committed migration.
Legacy labels & copy	<code>eval_runner</code> labels the vector retriever '(ChromaDB)' though it is Qdrant; the landing page mentions FAISS/BM25/Claude 3 Haiku; evaluator keeps a deprecated <code>NEED_CLARIFICATION</code> constant; config retains commented legacy blocks (E5 embeddings, ChromaDB, mmarco reranker) for rollback.
Thai legal documents not ingested	Documents/ PDFs (PDPA, Cybersecurity Act, ...) are not part of any ingestion path yet — the knowledge base currently contains only MITRE ATT&CK. Thai-law coverage arrives via OCR-per-upload (query-file) or the LLM's own knowledge.
Report retrieval is single-shot	<code>/generate-report</code> uses plain dual-query retrieval without the evaluator/follow-up loop, so reports on vague inputs rely on the model flagging gaps in <code>system_limitations</code> .
chat history unused	<code>chatContinue()</code> accepts a history parameter but discards it — each query is independent except during follow-up sessions.

Table 15-1: Issues and intentional trade-offs observed in the code (June 2026).

16. Appendix: Key File Index

Path	Purpose
rag_service/app/main.py	RAG FastAPI host; shared model loading; /query /resume /generate-report.
.../GraphRAG/config.py	All tunables: models, top-K, RRF, dual-query, DB endpoints.
.../GraphRAG/models.py	Pydantic entities/relationships.
.../GraphRAG/main.py	CLI: --ingest --test --agent --retrieve-only --local.
.../pipeline/agent_graph.py	LangGraph agent: state, nodes, edges, sessions, resume.
.../pipeline/chain.py	Linear LCEL pipeline (chain mode).
.../pipeline/evaluator.py	Context sufficiency judge, slots, fallback strategies.
.../pipeline/query_merger.py	Follow-up answer → MITRE-aligned rewrite.
.../pipeline/cross_lingual.py	Translation prompts, dual-query builder, language detection.
.../pipeline/router.py	GENERAL_EXPLANATION vs INCIDENT_ANALYSIS classifier.
.../pipeline/context_builder.py	Context assembly + generation prompt (confirmed facts).
.../pipeline/report_generator.py	CyberCaseReport structured output.
.../retrieval/vector_retriever.py	Qdrant hybrid (dense+sparse RRF) search.
.../retrieval/reranker.py	Cross-encoder rescoring.
.../retrieval/graph_retriever.py	Neo4j subgraph expansion + path queries.
.../retrieval/hybrid_retriever.py	retrieve() / retrieve_multi() fusion.
.../ingestion/stix_parser.py	STIX 2.1 → typed entities/relationships (+derived edges).
.../ingestion/graph_loader.py	Neo4j bulk load: constraints, UNWIND batches.
.../ingestion/vector_loader.py	Qdrant collections + hybrid embedding upserts.
.../evaluation/eval_runner.py	Benchmark CLI (retriever / generation / full).
.../evaluation/generate_eval_dataset.py	Neo4j-grounded dataset generator.
.../evaluation/crosslingual_benchmark.py	tRAG vs Thai-direct vs dual-query.
rag_service/finetune/ft_config.py	Fine-tune knobs (models, LoRA, paths).
rag_service/finetune/data/build_dataset.py	STIX → SFT jsonl builder.
rag_service/finetune/train/train_unsloth.py	QLoRA trainer (cloud GPU).
rag_service/finetune/compare/run_comparison.py	Base vs fine-tuned A/B harness.
backend/app/routers/rag.py	Gateway proxy routes incl. OCR query-file.
backend/app/services/typhoon_ocr_reader.py	Upload validation + Typhoon OCR.
frontend/src/lib/api.ts	Typed API client; base-URL normalization.
frontend/src/app/chat/page.tsx	Chat UI + follow-up session handling.

Path	Purpose
Mitre_ATT&CK Doc/schema_design.md	Original GraphRAG schema design document.

Table 16-1: Where to look first when modifying the system.